Redegal Chatbot + WebPhone VoIP

Documentacion Tecnica y Funcional Completa

Version: 1.0.0 **Fecha:** Febrero 2026 **Autor:** Redegal — A Smart Digital Company
Clasificacion: Confidencial - Uso Interno y Stakeholders

Tabla de Contenidos

1. Resumen Ejecutivo
2. Arquitectura del Sistema
3. Casos de Uso
4. Historias de Usuario
5. Guia de Instalacion desde Cero
6. Configuracion del Backend
7. Flujo de Trabajo por Unidad de Negocio (BU)
8. Canales de Comunicacion
9. Almacenamiento de Datos y Persistencia
10. Inteligencia Artificial y Aprendizaje
11. Concurrencia y Multi-Usuario
12. Gestion de Leads y Notificaciones
13. Funcionalidades del Dashboard
14. Integraciones
15. Seguridad
16. Guia de Despliegue en Produccion

---

1. Resumen Ejecutivo

1.1 Que es Redegal Chatbot

Redegal Chatbot es una plataforma premium de atencion al cliente que combina un **chatbot con inteligencia artificial multi-proveedor**, un **sistema de telefonia VoIP completo** (WebRTC + Click2Call) y un **dashboard profesional para agentes humanos**, todo integrable como widget embebible en cualquier sitio web.

El sistema esta disenado para la web corporativa de **Redegal**, consultora digital lider cotizada en BME Growth (RDG), y soporta las cuatro lineas de negocio de la compania: Boostic (SEO/Growth), Binnacle (Business Intelligence), Marketing Digital y Digital Tech.

1.2 Propuesta de Valor

Capacidad	Descripcion
IA Multi-Proveedor	Claude (Anthropic) como principal, Gemini (Google) como fallback gratuito, OpenAI como tercer nivel. Cambio automatico transparente.
Multi-Tenant	Soporte para multiples clientes con temas, configuraciones y API keys independientes.
4 Idiomas	Espanol, ingles, portugues y gallego con auto-deteccion del idioma del visitante.
VoIP Completo	Llamadas desde el navegador via WebRTC (Janus Gateway) y Click2Call con devolucion de llamada via SIP/Vozelia.
4 Lineas de Negocio	Routing inteligente, knowledge base y analytics segmentados por BU.
Dashboard Premium	SPA completa para agentes con inbox, wallboard en tiempo real, analytics y gestion de equipo.
CRM Universal	Conectores nativos para Salesforce, HubSpot, Zoho y Pipedrive.
Plugins E-Commerce	WordPress, Shopify y Magento 2 listos para instalar.
Aprendizaje Continuo	El bot mejora automaticamente con feedback de agentes y puntuaciones CSAT.

1.3 Diferenciadores Clave

1. **Arquitectura Shadow DOM**: El widget se inyecta sin conflictos CSS con la pagina anfitriona.
2. **RAG con pgvector**: Busqueda semantica en la base de conocimiento mediante embeddings vectoriales.
3. **Auto-scraping**: Actualizacion automatica de la knowledge base desde redegal.com cada 24 horas.
4. **Webhooks HMAC**: Notificaciones firmadas criptograficamente para integraciones seguras.
5. **Grabacion de llamadas**: Con transcripcion via IA y retencion configurable de 30 dias.

6. **Mensajeria proactiva**: Triggers inteligentes por tiempo en pagina, exit intent, abandono de carrito, etc.

---

2. Arquitectura del Sistema

2.1 Diagrama de Arquitectura de Alto Nivel

```
                                INTERNET
                                   |
                                   v
                        +-------------------+
                        |   Nginx Reverse   |
                        |   Proxy (SSL/TLS) |
                        +-------------------+
                                   |
                    +---------------+---------------+
                    |               |               |
                    v               v               v
              HTTP REST      WebSocket x3     Static Files
               /api/*       /ws/chat          /widget/*
                            /ws/agent         /dashboard/*
                            /ws/sip
                    |               |               |
                    +-------+-------+-------+-------+
                            |
                            v
                 +-----------------------+
                 |    Node.js Server     |
                 |    (Express + ws)     |
                 |    Puerto 9456        |
                 +-----------------------+
                  |    |    |    |    |
        +----------+    |    |    |    +----------+
        |               |    |    |               |
        v               v    |    v               v
  +-----------+  +--------+  |  +--------+  +-----------+
  | PostgreSQL|  |  Redis |  |  | Janus  |  |  Vozelia  |
  | 16 +      |  |  7     |  |  | WebRTC |  |  Cloud    |
  | pgvector  |  | (Alpine)|  |  | Gateway|  |  PBX      |
  +-----------+  +--------+  |  +--------+  +-----------+
```

```
                              |
                              v
                     +-------------------------+
                     |   Proveedores IA        |
                     |  Claude | Gemini |      |
                     |  OpenAI                 |
                     +-------------------------+
```

2.2 Vision General de Componentes

```
redegal-chatbot/
 |-- server/               # Backend Node.js 20 + Express +
WebSocket
 |   |-- config/           # SQL schema, knowledge YAML files
 |   |-- integrations/     # CRM connectors, webhook engine
 |   |-- middleware/        # Auth, CORS, CSRF, security headers
 |   |-- routes/           # REST API endpoints
 |   |-- services/         # AI, SIP, call recording, KB,
learning
 |   |-- state/            # Conversation FSM, session store
 |   |-- utils/            # DB, Redis, logger, i18n
 |   |-- ws/               # WebSocket handlers (chat, agent,
SIP)
 |   `-- public/           # Built widget + dashboard static
files
 |
 |-- widget/               # React 19 + TypeScript + Tailwind
 |   `-- src/              # IIFE bundle con Shadow DOM (227KB JS
+ 13KB CSS)
 |
 |-- dashboard/            # React 19 + TypeScript + Tailwind
 |   `-- src/              # Vite SPA para agentes (245KB JS +
29KB CSS)
 |
 |-- plugins/              # Plugins para CMS
 |   |-- wordpress/        # Plugin PHP completo con admin panel
 |   |-- shopify/          # Liquid snippet + settings schema
 |   `-- magento/          # Plantilla PHTML para Magento 2
 |
 |-- janus/                # Configuracion del Janus WebRTC
Gateway
```

```
|-- docker-compose.yml    # Orquestacion de contenedores
`-- .env                  # Variables de entorno
```

2.3 Stack Tecnologico

Capa	Tecnologia	Version	Proposito
Runtime	Node.js	20 LTS	Servidor backend
Framework HTTP	Express	4.x	API REST, middleware
WebSocket	ws	8.x	Chat en tiempo real, signaling SIP, agentes
Frontend Widget	React + TypeScript	19.x	Widget embebible IIFE
Frontend Dashboard	React + TypeScript	19.x	SPA para agentes humanos
Estilos	Tailwind CSS	4.x	Prefijo `rc-` para aislamiento
Base de Datos	PostgreSQL + pgvector	16	Datos persistentes + embeddings vectoriales
Cache/Pub-Sub	Redis	7 Alpine	Sesiones, pub/sub, rate limiting, colas
IA Principal	Claude (Anthropic)	claude-sonnet-4	Generacion de respuestas
IA Fallback	Gemini (Google)	2.5 Flash	Fallback gratuito
IA Terciario	OpenAI	GPT-4o-mini	Tercer nivel de fallback
Transcripcion	Whisper (OpenAI) / Gemini	1 / 2.0 Flash	Transcripcion de llamadas
VoIP WebRTC	Janus Gateway	Latest	Puente WebRTC-a-SIP
VoIP Click2Call	SIP UDP nativo	RFC 3261	Llamadas PSTN via Vozelia
PBX	Vozelia Cloud PBX	-	Centralita virtual
Email	Nodemailer	8.x	Notificaciones SMTP
Contenedores	Docker Compose	3.x	Orquestacion de servicios

Bundler	Vite	6.x	Build del widget y dashboard

2.4 Vision General del Schema de Base de Datos

```
+-------------------+     +------------------+     +-------------
------+
|     tenants       |     |   conversations  |     |     messages
|
|-------------------|     |------------------|     |-------------
------|
| id (UUID) PK      |<--->| tenant_id (FK)   |     | id (UUID) PK
|
| slug (UNIQUE)     |     | id (UUID) PK     |<--->|
conversation_id   |
| name              |     | visitor_id       |     | sender
(enum)     |
| domain            |     | language         |     | content
|
| api_key (UNIQUE)  |     | business_line    |     | message_type
|
| settings (JSONB)  |     | state (FSM)      |     | rich_content
|
+-------------------+     | agent_id (FK)    |     | attachments
|
         |                | tags[]           |     | read_at
|
         v                | priority         |     | metadata
(JSONB)  |
+-------------------+     | metadata (JSONB) |     +-------------
------+
|  widget_themes    |     +------------------+
|-------------------|              |
| tenant_id (FK)    |     +--------+--------+
| name              |     |                 |
| is_active         |     v                 v
| config (JSONB)    |  +----------+  +-----------+
+-------------------+  |  leads   |  |   calls   |
                       |----------|  |-----------|
+-------------------+  | name     |  | call_id   |
|     agents        |  | email    |  | status    |
|-------------------|  | phone    |  | duration  |
| id (UUID) PK      |  | company  |  | recording |
```

```
| username (UNIQUE) |  | score    |  | quality   |
| password_hash     |  | status   |  +-----------+
| display_name      |  | tags[]   |
| role (enum)       |  +----------+  +-----------------+
| languages[]       |                | call_recordings  |
| business_lines[]  |                |-----------------|
| sip_extension     |                | call_id          |
| status (enum)     |                | transcript       |
| max_concurrent    |                | recording_path   |
+-------------------+                | retention_note   |
                                     +-----------------+

+---------------------+  +-------------------+  +----------------
-----+
| knowledge_entries   |  |    webhooks       |  |
analytics_events    |
|---------------------|  |-------------------|  |----------------
-----|
| business_line       |  | url               |  | event_type
|
| language            |  | events[]          |  | conversation_id
|
| title               |  | secret (HMAC)     |  | visitor_id
|
| content             |  | is_active         |  | agent_id
|
| embedding (vector)  |  | failure_count     |  | business_line
|
| source              |  +-------------------+  | data (JSONB)
|
+---------------------+                         +----------------
-----+

+---------------------+  +---------------------+  +--------------
----+
| response_feedback   |  | learned_responses   |  |
canned_responses |
|---------------------|  |---------------------|  |--------------
----|
| visitor_message     |  | visitor_pattern     |  | shortcut
|
| ai_response         |  | ideal_response      |  | title
|
| feedback (good/bad) |  | embedding (vector)  |  | content
```

```
|
| corrected_response  |  | confidence          |  | language
|
| csat_rating         |  | is_active           |  | business_line
|
| auto_learned        |  +---------------------+  | usage_count
|
+---------------------+                           +--------------
----+

+------------------+  +------------------------+  +--------------
----+
|   callbacks      |  | conversation_transfers |  |
kb_scrape_log   |
|------------------|  |------------------------|  |--------------
----|
| phone            |  | from_agent_id          |  | url
|
| scheduled_date   |  | to_agent_id            |  | content_hash
|
| time_slot        |  | reason                 |  | entries_added
|
| status           |  +------------------------+  | status
|
+------------------+                               +-------------
-----+
```

Extensiones PostgreSQL utilizadas:

- `uuid-ossp` : Generacion de UUIDs v4
- `pgcrypto` : Generacion de API keys aleatorias
- `vector` : Embeddings vectoriales para busqueda semantica (pgvector)

---

3. Casos de Uso

CU-01: Visitante Inicia Chat y Recibe Respuesta de IA

Actor: Visitante de la web **Precondicion:** Widget cargado en la pagina **Flujo principal:**

1. El visitante hace clic en el boton flotante del widget.

2. El widget abre conexion WebSocket a `/ws/chat`.
3. El servidor envia `connected` con estado de horario comercial.
4. El visitante escribe un mensaje.
5. El servidor detecta automaticamente el idioma del primer mensaje.
6. El sistema detecta la linea de negocio por keywords (SEO, BI, etc.).
7. Se crea una conversacion en la base de datos con estado `chat_active`.
8. El router consulta la knowledge base (full-text + vector search + respuestas aprendidas).
9. Se genera respuesta via el proveedor de IA activo (cadena de fallback: Gemini -> Claude -> OpenAI).
10. El bot puede adjuntar rich content (quick replies, cards, botones) segun la intencion detectada.
11. Se almacena la respuesta del bot para posterior revision/aprendizaje.
12. El visitante recibe la respuesta en su idioma.

Postcondicion: Conversacion activa, metricas registradas, webhook `conversation.started` disparado.

CU-02: Visitante Solicita Callback (Click2Call)

Actor: Visitante de la web **Precondicion:** Horario comercial activo, visitante en conversacion **Flujo principal:**

1. El visitante solicita una llamada proporcionando su numero de telefono.
2. El servidor valida el numero (minimo 6 digitos).
3. Se crea un registro de llamada en la tabla `calls` con estado `ringing`.
4. El sistema verifica si el cliente SIP esta registrado en Vozelia.
5. **Si SIP esta activo**: Se origina una llamada INVITE al telefono del visitante.
6. Cuando el visitante responde, se ejecuta REFER para transferir la llamada a las extensiones de agentes (secuencialmente: 107, 158, 105).
7. La PBX puentea la llamada visitante <-> agente.
8. **Si SIP no esta activo**: Se encola la llamada para devolucion manual.
9. Se notifica a todos los agentes conectados via Redis pub/sub.
10. Se envia email de notificacion con el numero del visitante.

Postcondicion: Llamada conectada o encolada, webhook `call.started` disparado.

CU-03: Visitante Realiza Llamada desde el Navegador (WebRTC)

Actor: Visitante de la web **Precondicion:** Horario comercial, navegador compatible con WebRTC **Flujo principal:**

1. El visitante selecciona "Llamar desde el navegador" en el widget.
2. El servidor genera un `callId` unico y registra la llamada como `setup`.
3. Se envia al widget la configuracion de Janus: URL del WebSocket, credenciales SIP y servidores STUN.
4. El widget se conecta al Janus Gateway via WebSocket (`ws://janus:8188`).
5. Janus registra una extension SIP temporal en la PBX de Vozelia.
6. Se inicia la llamada SIP hacia la extension del agente objetivo.
7. Janus maneja la negociacion SDP (offer/answer) y el relay ICE.
8. El audio se transmite via WebRTC (navegador <-> Janus <-> SIP <-> telefono del agente).
9. Janus graba la llamada en formato `.mjr` en `/tmp/janus-recordings/`.
10. Al colgar, se actualiza la base de datos con duracion y se importa la grabacion.

Postcondicion: Llamada completada, grabacion disponible, webhook `call.ended` disparado.

CU-04: Agente Acepta Conversacion Escalada

Actor: Agente humano del equipo Redegal **Precondicion:** Agente autenticado y conectado al dashboard via `/ws/agent` **Flujo principal:**

1. Un visitante solicita hablar con un agente humano.
2. La conversacion cambia de estado `chat_active` a `chat_waiting_agent` (FSM).
3. Se notifica a todos los agentes conectados via Redis `queue:new`.
4. Se envia email de escalacion al equipo correspondiente.
5. El agente ve la conversacion en su cola de inbox.
6. El agente acepta la conversacion (claim atomico con `WHERE agent_id IS NULL` para evitar race conditions).
7. El servidor genera un resumen IA de la conversacion previa para contexto rapido del agente.

8. Se envia al agente el historial completo de mensajes y el contexto de pagina del visitante.
9. Se notifica al visitante que un agente se ha unido.
10. Se elimina la conversacion de la cola del resto de agentes.
11. El agente puede usar atajos (`/saludo` , `/precio`) que se expanden a respuestas predefinidas.

Postcondicion: Conversacion asignada al agente, estado `chat_active` , webhook `agent.assigned` disparado.

CU-05: Captura y Scoring de Lead

Actor: Sistema automatico **Precondicion:** Visitante proporciona datos de contacto (formulario o conversacion) **Flujo principal:**

1. El visitante completa el formulario de lead (nombre obligatorio, email, telefono, empresa).
2. Se valida el formato del email y la longitud del nombre.
3. Se guarda el lead en la base de datos con la linea de negocio e idioma de la conversacion.
4. El sistema ejecuta el algoritmo de scoring deterministico:

Dimension	Puntos
Email proporcionado	+20
Empresa proporcionada	+15
Telefono proporcionado	+10
Linea de negocio identificada	+10
Mensajes intercambiados (x2 cada, max 20)	+2-20
Escalacion a agente humano	+15
Solicitud de llamada VoIP	+10
Maximo	**100**

5. Se despacha el lead al CRM configurado (Salesforce, HubSpot, etc.).
6. Se dispara webhook `lead.created` .
7. Se registra el evento de analytics `lead_captured` .

Postcondicion: Lead almacenado y puntuado, CRM actualizado, webhook notificado.

CU-06: Formulario Offline Fuera de Horario

Actor: Visitante fuera de horario comercial **Precondicion:** Fuera del horario configurado (L-V 09:00-19:00 Europe/Madrid) **Flujo principal:**

1. El visitante intenta comunicarse fuera de horario.
2. El widget muestra un mensaje indicando el horario de atencion.
3. Se presenta un formulario offline con campos: nombre, email, telefono (opcional), mensaje.
4. Los datos se guardan como lead con metadato `source: offline_form`.
5. Se ejecuta scoring y despacho a CRM normalmente.
6. Se dispara webhook `lead.created` con source `offline_form`.

CU-07: Grabacion y Transcripcion de Llamadas

Actor: Sistema automatico / Supervisor **Precondicion:** Llamada en curso (Click2Call o WebRTC) **Flujo principal:**

1. Al iniciar una llamada Click2Call, se envia header SIP `Record: on` a la PBX.
2. Para llamadas WebRTC, Janus activa grabacion automatica (`record: true).
3. Las grabaciones se almacenan en `/server/recordings/` (Click2Call) o `/janus/recordings/` (WebRTC).
4. Al finalizar la llamada, se registra la grabacion en la tabla `call_recordings`.
5. Un supervisor puede solicitar la transcripcion de una llamada.
6. Se intenta transcribir primero con Whisper (OpenAI) y como fallback con Gemini 2.0 Flash.
7. La transcripcion incluye etiquetas de hablante (Visitor/Agent).
8. El cron de limpieza elimina grabaciones de mas de 30 dias (cada 6 horas), conservando metadatos hasta 90 dias.

CU-08: Busqueda en la Base de Conocimiento

Actor: Visitante de la web **Precondicion:** Widget abierto con conversacion activa **Flujo principal:**

1. El visitante envia un mensaje tipo `search_kb` o simplemente chatea.
2. El sistema ejecuta busqueda en tres niveles:
   - **Full-text search** en PostgreSQL con `to_tsvector`.
   - **Busqueda semantica** via pgvector (cosine similarity > 0.5).
   - **Respuestas aprendidas** de interacciones previas exitosas (similarity > 0.6).
3. Los resultados se combinan y alimentan el prompt del sistema de IA.
4. Se retornan hasta 8 resultados con titulo, contenido (truncado a 300 chars), categoria y BU.

CU-09: Mensajeria Proactiva por Triggers

Actor: Sistema automatico **Precondicion:** Visitante navegando sin conversacion activa **Flujo principal:**

1. El widget envia contexto de pagina periodicamente (`page_context`).
2. El servidor evalua 6 tipos de triggers:

Trigger	Condicion	Cooldown	Prioridad
`time_on_page`	>= 30 segundos en la pagina	5 min	1
`pricing_page`	URL contiene /precio, /pricing, /plans	10 min	3
`exit_intent`	Cursor sale de la ventana	15 min	2
`return_visitor`	>= 2 visitas	1 hora	1
`cart_abandon`	Carrito con valor > 0 y > 60s en pagina	20 min	4
`idle_on_form`	Interaccion con formulario + 20s inactivo	5 min	2

3. Solo se dispara el trigger de mayor prioridad (para no saturar al visitante).
4. Los cooldowns se gestionan en Redis por visitor + trigger.
5. Los mensajes estan localizados en los 4 idiomas soportados.

CU-10: Integracion con CRM

Actor: Sistema automatico **Precondicion:** CRM configurado en la tabla `config` con clave `crm_integrations` **Flujo principal:**

1. Se crea un lead o se cierra una conversacion.
2. El sistema obtiene la configuracion de integraciones CRM activas.
3. Para cada CRM habilitado, se instancia el adaptador correspondiente.
4. **En `lead_created`** : Se crea contacto/lead y opcionalmente un deal/oportunidad.
5. **En `conversation_closed`** : Se registra la actividad si `logActivities` esta habilitado.
6. Los tokens OAuth (Salesforce, Zoho) se cachean en Redis con TTL de 3500 segundos.

CU-11: Notificaciones via Webhooks

Actor: Sistema externo suscrito **Precondicion:** Webhook configurado en la tabla `webhooks` con eventos suscritos **Flujo principal:**

1. Ocurre un evento en el sistema (ej: `lead.created`).
2. Se consultan los webhooks activos suscritos a ese evento con `failure_count < 50` .
3. Se valida que la URL destino no apunte a redes internas (prevencion SSRF).
4. Se construye el payload JSON con evento, datos y timestamp.
5. Si el webhook tiene `secret` , se genera firma HMAC-SHA256 en el header `X-Webhook-Signature` .
6. Se envia el POST con timeout de 10 segundos.
7. Si falla, se reintenta hasta 5 veces con backoff exponencial (5s, 10s, 20s, 40s, 80s).
8. Se registra la entrega en `webhook_deliveries` para auditoria.

---

4. Historias de Usuario

4.1 Visitante (Usuario de la Web)

ID	Historia de Usuario

V-01	Como visitante, quiero abrir un chat con un clic para obtener informacion sobre los servicios de Redegal sin tener que buscar por toda la web.
V-02	Como visitante, quiero que el chatbot me responda en mi idioma automaticamente para comunicarme de forma natural.
V-03	Como visitante, quiero poder cambiar el idioma manualmente (ES/EN/PT/GL) para asegurarme de que la comunicacion es en mi idioma preferido.
V-04	Como visitante, quiero hablar con un agente humano cuando el bot no puede resolver mi consulta, para obtener una respuesta personalizada.
V-05	Como visitante, quiero poder llamar desde el navegador sin instalar nada, para hablar directamente con el equipo comercial.
V-06	Como visitante, quiero solicitar que me devuelvan la llamada proporcionando mi telefono, para no tener que esperar en linea.
V-07	Como visitante, quiero dejar mis datos de contacto (lead) cuando me interesa un servicio, para que el equipo comercial me contacte.
V-08	Como visitante, quiero recibir mensajes enriquecidos (tarjetas, botones, quick replies) que me faciliten la navegacion del chat.
V-09	Como visitante fuera de horario, quiero poder dejar un formulario offline con mi consulta, para que me atiendan al dia siguiente.
V-10	Como visitante, quiero poder valorar mi experiencia (CSAT 1-5 estrellas) al finalizar, para contribuir a mejorar el servicio.
V-11	Como visitante, quiero buscar directamente en el centro de ayuda del chatbot, para encontrar respuestas sin esperar.
V-12	Como visitante, quiero descargar la transcripcion de mi conversacion, para tener un registro de lo hablado.
V-13	Como visitante recurrente, quiero que el chat recuerde mi conversacion anterior, para no tener que repetir informacion.

4.2 Agente (Miembro del Equipo Redegal)

ID	Historia de Usuario
A-01	Como agente, quiero ver una cola de conversaciones pendientes en mi inbox, para saber que visitantes necesitan atencion.
A-02	Como agente, quiero aceptar conversaciones de la cola y ver un resumen IA del contexto previo, para atender rapidamente sin leer todo el historial.

A-03	Como agente, quiero usar atajos de teclado (`/saludo` , `/precio`) que se expandan en respuestas predefinidas, para ser mas eficiente.
A-04	Como agente, quiero transferir una conversacion a otro agente con un motivo, para derivar al especialista adecuado.
A-05	Como agente, quiero escribir notas internas (visibles solo para agentes) en una conversacion, para compartir contexto con el equipo.
A-06	Como agente, quiero aceptar o rechazar llamadas entrantes desde el dashboard, para gestionar mis interacciones de voz.
A-07	Como agente, quiero ver el indicador de escritura del visitante, para saber cuando esta redactando un mensaje.
A-08	Como agente, quiero marcar mensajes como leidos para que el visitante vea confirmacion de lectura, mejorando la experiencia.
A-09	Como agente, quiero anadir etiquetas y prioridad a las conversaciones, para organizarlas y facilitar el seguimiento.
A-10	Como agente, quiero obtener sugerencias de respuesta generadas por IA, para responder mas rapido y de forma consistente.
A-11	Como agente, quiero revisar respuestas del bot y marcarlas como buenas/malas, para entrenar al sistema.

4.3 Administrador / Supervisor

ID	Historia de Usuario
M-01	Como supervisor, quiero ver un wallboard en tiempo real con metricas globales (llamadas activas, chats en cola, CSAT medio), para monitorizar el rendimiento del equipo.
M-02	Como administrador, quiero gestionar respuestas predefinidas (canned responses) agrupadas por idioma, BU y categoria, para estandarizar la comunicacion.
M-03	Como administrador, quiero configurar webhooks con eventos especificos y secretos HMAC, para integrar el chatbot con sistemas externos.
M-04	Como administrador, quiero personalizar completamente el tema del widget (colores, tipografia, layout, funcionalidades), para adaptarlo a la marca de cada cliente.
M-05	Como supervisor, quiero ver graficas de analytics (conversaciones por dia, leads por BU, tiempos de respuesta), para tomar decisiones basadas en datos.

M-06	Como administrador, quiero crear y gestionar usuarios agentes con roles (agent, supervisor, admin), para controlar el acceso.
M-07	Como supervisor, quiero acceder al historial de llamadas con grabaciones y transcripciones, para auditar la calidad del servicio.
M-08	Como supervisor, quiero monitorizar llamadas en curso (listen-in) sin que el visitante lo sepa, para supervision de calidad.
M-09	Como administrador, quiero configurar las integraciones CRM desde el dashboard, para conectar el chatbot con nuestro sistema de ventas.
M-10	Como administrador, quiero ver el registro de entregas de webhooks con estados y errores, para diagnosticar problemas de integracion.

4.4 Desarrollador (Integracion / Despliegue)

ID	Historia de Usuario
D-01	Como desarrollador, quiero instalar el widget con un unico tag `<script>` y un objeto de configuracion, para integrarlo rapidamente en cualquier sitio.
D-02	Como desarrollador WordPress, quiero instalar un plugin con panel de configuracion en el admin, para activar el chatbot sin tocar codigo.
D-03	Como desarrollador Shopify, quiero usar un snippet Liquid con configuracion desde el Theme Editor, para una integracion nativa.
D-04	Como desarrollador, quiero recibir webhooks HMAC-firmados de los eventos del chatbot, para integrarlos con nuestro backend de forma segura.
D-05	Como DevOps, quiero desplegar todo el stack con `docker compose up --build`, para simplificar el proceso de produccion.
D-06	Como desarrollador, quiero acceder a una API REST documentada para crear agentes, consultar analytics y gestionar leads programaticamente.

---

5. Guia de Instalacion desde Cero

5.1 Prerequisitos

Requisito	Version Minima	Notas
Docker	24.x	Con Docker Compose V2

Node.js	20 LTS	Solo para desarrollo local
npm	10.x	Incluido con Node.js 20
Git	2.x	Para clonar el repositorio

5.2 Clonar el Repositorio

```
git clone https://github.com/jorgevazquez-vagojo/redegal-chatbot.git
cd redegal-chatbot
```

5.3 Configurar Variables de Entorno

```
cp .env.example .env
```

Editar `.env` con los valores necesarios (ver seccion 6 para detalle completo):

```
# Base de datos
POSTGRES_PASSWORD=tu_password_seguro

# Redis
REDIS_PASSWORD=tu_redis_password

# IA (al menos uno obligatorio)
AI_PROVIDER=gemini
GEMINI_API_KEY=tu_api_key_gemini

# Seguridad
JWT_SECRET=cadena_aleatoria_de_32_caracteres
WIDGET_API_KEY=tu_widget_api_key

# SIP / Vozelia (opcional)
SIP_DOMAIN=cloudpbx1584.vozelia.com
SIP_EXTENSION=108
SIP_PASSWORD=tu_sip_password
```

5.4 Instalar Dependencias (Desarrollo Local)

```
npm install
```

5.5 Inicializar la Base de Datos

La inicializacion es automatica con Docker Compose. El archivo `server/config/init.sql` se ejecuta al crear el contenedor de PostgreSQL por primera vez, creando todas las tablas, indices, extensiones y datos por defecto (tenant `redegal` + tema default).

5.6 Iniciar con Docker Compose (Recomendado)

```
# Build del widget y dashboard
npm run build

# Levantar todos los servicios
docker compose up --build -d
```

Servicios levantados:

Servicio	Puerto	Descripcion
`postgres`	5432 (local)	PostgreSQL 16 + pgvector
`redis`	6379 (local)	Redis 7 Alpine
`server`	9456 (local)	Aplicacion Node.js
`janus`	8088, 8188, 7088	Janus WebRTC Gateway

5.7 Desarrollo Local (Sin Docker para el Server)

```
# Levantar solo DB y cache
docker compose up -d postgres redis

# Servidor en modo desarrollo (hot reload)
npm run dev:server
```

```
# Widget en modo desarrollo (en otra terminal)
npm run dev:widget

# Dashboard en modo desarrollo (en otra terminal)
npm run dev:dashboard
```

5.8 Crear el Primer Usuario Agente

```
node scripts/create-agent.js admin admin123 "Administrador"
"es,gl,en,pt" "boostic,binnacle,marketing,tech"
```

Parametros:

1. `username` - Nombre de usuario para login
2. `password` - Contrasena
3. `displayName` - Nombre visible en el chat
4. `languages` - Idiomas separados por coma
5. `businessLines` - Lineas de negocio separadas por coma

5.9 Verificar la Instalacion

```
# Health check
curl http://localhost:9456/health

# Probar el widget
open http://localhost:9456/widget/test.html

# Acceder al dashboard
open http://localhost:9456/dashboard
```

5.10 Build de Produccion

```
# Compilar widget (IIFE + Shadow DOM)
npm -w widget run build
# Resultado: widget/dist/ (~227KB JS + 13KB CSS)

# Compilar dashboard (Vite SPA)
npm -w dashboard run build
# Resultado: dashboard/dist/ (~245KB JS + 29KB CSS)
```

---

6. Configuracion del Backend

6.1 Variables de Entorno Completas

Base de Datos

Variable	Obligatoria	Default	Descripcion
`POSTGRES_HOST`	Si	`localhost`	Hostname del servidor PostgreSQL
`POSTGRES_PORT`	No	`5432`	Puerto de PostgreSQL
`POSTGRES_DB`	No	`redegal_chatbot`	Nombre de la base de datos
`POSTGRES_USER`	No	`redegal`	Usuario de PostgreSQL
`POSTGRES_PASSWORD`	**Si**	-	Contrasena de PostgreSQL

Redis

Variable	Obligatoria	Default	Descripcion
`REDIS_HOST`	No	`localhost`	Hostname del servidor Redis
`REDIS_PASSWORD`	Prod: Si	`""`	Contrasena de Redis

Proveedores de IA

Variable	Obligatoria	Default	Descripcion

`AI_PROVIDER`	No	`gemini`	Proveedor principal: `gemini`, `anthropic`, `openai`
`GEMINI_API_KEY`	Condicional	-	API Key de Google Gemini (gratuita con limite)
`ANTHROPIC_API_KEY`	Condicional	-	API Key de Anthropic Claude
`OPENAI_API_KEY`	Condicional	-	API Key de OpenAI

Nota: Al menos un proveedor de IA debe estar configurado. La cadena de fallback es: proveedor principal -> gemini -> anthropic -> openai.

Servidor

Variable	Obligatoria	Default	Descripcion
`PORT`	No	`9456`	Puerto del servidor HTTP
`BIND_HOST`	No	`127.0.0.1`	IP de bind (usar `0.0.0.0` en Docker)
`NODE_ENV`	No	`development`	Entorno: `development` o `production`
`JWT_SECRET`	**Prod: Si**	Auto-generado	Secreto para firmar tokens JWT (12h expiry)
`WIDGET_API_KEY`	No	`dev-widget-key`	API Key para autenticar el widget

SIP / Vozelia (Click2Call)

Variable	Obligatoria	Default	Descripcion
`SIP_DOMAIN`	No	-	Dominio del servidor SIP (ej: `cloudpbx1584.vozelia.com`
`SIP_EXTENSION`	No	-	Extension SIP del chatbot (ej: `108`)
`SIP_PASSWORD`	No	-	Contrasena de la extension SI
`CLICK2CALL_CALLERID_NAME`	No	`Lead Web`	Nombre CallerID que ven los agentes
`CLICK2CALL_EXTENSIONS`	No	`107`	Extensiones de agentes separadas por coma (ring group)

Janus WebRTC Gateway

Variable	Obligatoria	Default	Descripcion
`JANUS_URL`	No	`http://janus:8088/janus`	URL REST API de Janus
`JANUS_WS_URL`	No	`ws://janus:8188`	WebSocket interno de Janus
`JANUS_ADMIN_URL`	No	`http://janus:7088/admin`	API de administracion de Janus
`JANUS_ADMIN_SECRET`	No	`janus-admin-secret`	Secret del admin API
`JANUS_PUBLIC_WS`	No	`ws://localhost:8188`	URL publica del WebSocket de Janus (visible desde el widget)
`SERVER_PUBLIC_IP`	No	-	IP publica del servidor (para NAT traversal de Janus)

CORS y Seguridad

Variable	Obligatoria	Default	Descripcion
`ALLOWED_ORIGINS`	Prod: Si	`http://localhost:9456`	Origenes permitidos separados por coma

Horario Comercial

Variable	Obligatoria	Default	Descripcion
`TIMEZONE`	No	`Europe/Madrid`	Zona horaria para horario comercial
`BUSINESS_HOURS_START`	No	9	Hora de inicio (formato 24h)

`BUSINESS_HOURS_END`	No	`19`	Hora de fin (formato 24h)

Email / SMTP

Variable	Obligatoria	Default	Descripcion
`SMTP_HOST`	No	-	Servidor SMTP
`SMTP_PORT`	No	`587`	Puerto SMTP (587=STARTTLS, 465=SSL)
`SMTP_USER`	No	-	Usuario SMTP
`SMTP_PASSWORD`	No	-	Contrasena SMTP
`SMTP_FROM`	No	`chatbot@redegal.com`	Direccion remitente
`NOTIFICATION_EMAIL`	No	-	Email(s) destino de notificaciones

Branding

Variable	Obligatoria	Default	Descripcion
`PRIMARY_COLOR`	No	`#007fff`	Color primario del widget

6.2 Modelos de IA Utilizados

Proveedor	Modelo	Uso	Coste
Anthropic	`claude-sonnet-4-20250514`	Respuestas chat (principal)	Pago por token
Google	`gemini-2.5-flash`	Respuestas chat (fallback gratuito)	Gratis (20 req/dia)
OpenAI	`gpt-4o-mini`	Respuestas chat (tercer nivel)	Pago por token
OpenAI	`whisper-1`	Transcripcion de llamadas	Pago por minuto
Google	`gemini-2.0-flash`	Transcripcion de llamadas (fallback)	Pago por token

Google	`text-embedding-004`	Embeddings vectoriales (768 dim)	Gratis

6.3 Configuracion de Janus WebRTC Gateway

El directorio `janus/` contiene los archivos de configuracion:

Archivo	Descripcion
`janus.jcfg`	Configuracion principal (NAT, debug)
`janus.transport.http.jcfg`	REST API en puerto 8088
`janus.transport.websockets.jcfg`	WebSocket en puerto 8188
`janus.plugin.sip.jcfg`	Plugin SIP para llamadas VoIP

Puertos necesarios:

- `8088` TCP: REST API (server -> Janus)
- `8188` TCP: WebSocket (widget -> Janus)
- `7088` TCP: Admin API
- `10000-10200` UDP: Puertos RTP para media

---

7. Flujo de Trabajo por Unidad de Negocio (BU)

7.1 Las 4 Lineas de Negocio

ID	Nombre	Descripcion	Color	Icono
`boostic`	Boostic - SEO & Growth	Posicionamiento, analitica web y CRO	`#3B82F6`	chart-line
`binnacle`	Binnacle - Business Intelligence	Dashboards, datos y reporting	`#8B5CF6`	chart-bar
`marketing`	Marketing Digital	SEM, Social Media y campanas	`#10B981`	megaphone
`tech`	Digital Tech	Desarrollo web, apps y e-commerce	`#F59E0B`	code

7.2 Auto-Deteccion de BU

La deteccion se realiza en dos niveles:

Por URL de la pagina (contexto de navegacion):

```
/seo, /boostic, /growth, /analytics     -> boostic
/bi, /binnacle, /dashboard, /dato       -> binnacle
/marketing, /publicidad, /ads, /social   -> marketing
/desarrollo, /tech, /ecommerce, /shopify -> tech
```

Por contenido del mensaje (keywords):

BU	Keywords
boostic	seo, posicionamiento, growth, analytics, organico, trafico, cro, conversion, keywords, crawl
binnacle	bi, business intelligence, dashboard, datos, data, warehouse, bigquery, looker, kpi, etl, reporting
marketing	sem, publicidad, ads, social media, redes sociales, campanas, email marketing, facebook, google ads, tiktok
tech	desarrollo, web, app, aplicacion, e-commerce, ecommerce, magento, shopify, programacion, api, cloud, integracion

7.3 Routing y Asignacion de Agentes

1. La BU detectada se almacena en la conversacion (`business_line`).
2. Al escalar a agente humano, se buscan agentes que tengan esa BU en su array `business_lines[]` .
3. Los agentes tambien se filtran por idioma (`languages[]`).
4. En la cola de llamadas, cada BU tiene su propia sorted set en Redis (`queue-boostic` , `queue-binnacle` , etc.).

7.4 Knowledge Base por BU

Cada BU tiene su propio archivo YAML en `server/config/knowledge/` :

```
server/config/knowledge/
 |-- general.yaml    # Informacion corporativa comun
 |-- boostic.yaml    # Conocimiento especifico de SEO/Growth
```

```
|-- binnacle.yaml   # Conocimiento especifico de BI
|-- marketing.yaml  # Conocimiento especifico de Marketing
`-- tech.yaml       # Conocimiento especifico de Tech
```

Al generar una respuesta, se combinan las entradas `general` + la BU especifica detectada.

7.5 Analytics por BU

Todos los eventos de analytics incluyen el campo `business_line`, permitiendo:

- Conversaciones por BU
- Leads por BU
- Llamadas por BU
- CSAT por BU
- Conversion rate por BU

---

8. Canales de Comunicacion

8.1 Sistema de Chat

8.1.1 Arquitectura de IA Multi-Proveedor con Fallback

```
                 Mensaje del Visitante
                          |
                          v
             +---------------------+
             |   Router (router.js) |
             +---------------------+
             | 1. Detectar BU       |
             | 2. Buscar KB (FTS)   |
             | 3. Vector search     |
             | 4. Learned responses |
             | 5. Build prompt      |
             +----------+----------+
                          |
                          v
             +---------------------+
             |  AI Complete (ai.js) |
```

```
+--------------------+
     |         |         |
     v         v         v
+--------+ +--------+ +--------+
| Gemini | | Claude | | OpenAI |
|(Primary)| |(Fallb) | |(Fallb) |
+--------+ +--------+ +--------+
     |         |         |
     +---------+---------+
           |
     Respuesta generada
           |
           v
+--------------------+
| Rich Content Builder |
| (cards, buttons,     |
|  quick replies)      |
+--------------------+
           |
           v
 Mensaje al Visitante
```

8.1.2 Mensajes Enriquecidos (Rich Messages)

El sistema genera automaticamente rich content segun la intencion detectada:

Intencion	Tipo de Rich Content	Ejemplo
Pregunta sobre servicios	`quick_replies`	Botones con las 4 BUs
Consulta especifica de BU	`card`	Tarjeta con titulo + CTA (Contactar experto, Dejar datos)
Pregunta sobre precios	`buttons`	Hablar con agente / Solicitar presupuesto / Llamar

Tipos de mensaje soportados:

```
text | image | file | audio | card | carousel | buttons |
quick_reply | system
```

8.1.3 Auto-Deteccion de Idioma

La deteccion se realiza sobre el primer mensaje del visitante usando analisis de patrones linguisticos. Idiomas soportados:

Codigo	Idioma	Deteccion
`es`	Espanol	Patrones hispanicos
`en`	Ingles	Patrones anglosajones
`pt`	Portugues	Patrones lusitanos
`gl`	Gallego	Patrones gallegos

8.1.4 Maquina de Estados de Conversacion (FSM)

```
                                 start
                                   |
                                   v
+-------------------+    +-------------------+
|    chat_idle      |--->|   chat_active     |<---------+
+-------------------+    +-------------------+          |
                         |  |  |              |          |
                escalate |  |  | request_call |   cancel |
                         v  |  v              |   timeout|
          +-----------------+ +-----------------+      |
          | chat_waiting_   | | escalation_     |------+
          | agent           | | pending         |
          +-----------------+ +-----------------+
               |                    |
          agent_accept          call_start
               |                    |
               |              +-----------------+
               |              | call_connecting  |
               |              +-----------------+
               |                    |
               |               connected
               |                    |
               |              +-----------------+
              +               | call_active     |
               |              +-----------------+
               |                    |
               |                 hangup
               |                    |
               |              +-----------------+
               |              | call_ended      |
```

```
                  |                +------------------+
                  |                       |
                  |                resume_chat
                  |                       |
                  +------->+<-----------+
                           |
                         close
                           |
                           v
                  +------------------+
                  |     closed       |
                  +------------------+
```

8.1.5 Escalacion a Agente Humano

1. Visitante solicita escalacion (boton o mensaje).
2. Se verifica disponibilidad de agentes por idioma y BU.
3. Si no hay agentes disponibles, se informa al visitante.
4. Si hay agentes, la conversacion transiciona a `chat_waiting_agent`.
5. Se notifica a todos los agentes via Redis `queue:new`.
6. Se envia email de notificacion con contexto de los ultimos 5 mensajes.
7. El primer agente que acepta reclama la conversacion (claim atomico).
8. Se genera resumen IA automatico para el agente.

8.1.6 Takeover y Handoff

- **Takeover**: El agente puede tomar el control en cualquier momento aceptando una conversacion en espera.
- **Handoff**: El agente puede transferir a otro agente indicando un motivo. El destinatario recibe el historial completo y una notificacion.

8.2 Llamadas WebRTC (Navegador)

8.2.1 Arquitectura: Widget -> Janus Gateway -> Vozelia PBX

```
+----------+     WebSocket      +---------+       SIP        +---
------+
|  Widget  |<================>|  Janus   |<===============>|
Vozelia |
|  (React) |    (wss:8188)     | Gateway  |   (UDP:5060)     |
Cloud   |
|          |                   |          |                  |
```

```
PBX    |
+----------+                  +---------+                  +---
------+
     |                              |
|
     |  getUserMedia()              |
|
     |  (microfono)                 |
|
     |                              |
|
     |  WebRTC Offer/Answer         |    SIP INVITE/200 OK
|
     |  ICE Candidates              |    RTP Audio
|
     |<---------------------------->|<---------------------------
>|
     |                              |
|
     |        Audio bidireccional via WebRTC
|
     |
<==========================================================>|
                          Janus actua como proxy
```

8.2.2 Flujo de Llamada Paso a Paso

1. **Solicitud**: Visitante pulsa "Llamar" -> `request_webrtc_call` via WebSocket.
2. **Setup**: Servidor genera `callId`, determina extension destino y responde con `webrtc_ready` (credenciales Janus + SIP + STUN servers).
3. **Conexion Janus**: Widget conecta a Janus via WebSocket, crea sesion y adjunta plugin SIP.
4. **Registro SIP**: Janus registra la extension del chatbot en Vozelia.
5. **Llamada**: Janus envia SIP INVITE a la extension del agente. Se negocian codecs (PCMU/PCMA).
6. **Media**: Audio fluye: Navegador <-(WebRTC)-> Janus <-(RTP)-> Vozelia <-(PSTN/SIP)-> Telefono agente.
7. **Grabacion**: Janus graba ambos lados del audio en archivos `.mjr`.
8. **Colgado**: Cualquiera de los dos extremos puede colgar. Se registra duracion y se importa grabacion.

8.2.3 Servidores STUN Configurados

```
iceServers: [
  { urls: 'stun:stun.l.google.com:19302' },
  { urls: 'stun:stun1.l.google.com:19302' },
]
```

8.2.4 Grabacion via Janus

- Las grabaciones se almacenan en `/tmp/janus-recordings/` dentro del contenedor Janus.
- El volumen esta mapeado a `./janus/recordings` en el host.
- El servidor importa las grabaciones post-llamada a `/server/recordings/`.
- Formatos: `.mjr` (nativo Janus), `.wav` (post-procesado con `janus-pp-rec`).

8.3 Click2Call Callback

8.3.1 Arquitectura: Server -> SIP INVITE -> Phone -> REFER -> Agent

```
+----------+     1. request_call     +----------+
|  Widget  |=======================>|  Server  |
| (visitor)|                         | (Node.js)|
+----------+                         +----------+
                                          |
                                    2. SIP REGISTER
                                    (si no registrado)
                                          |
                                          v
                                    +----------+
                                    | Vozelia  |
                                    | Cloud    |
                                    |   PBX    |
                                    +----------+
                                          |
                                    3. SIP INVITE
                                    (al telefono del visitante)
                                          |
```

```
                                        v
                                  +----------+
                                  | Telefono |
                                  | Visitante|
                                  +----------+
                                        |
                                  4. Visitante responde
                                  (200 OK)
                                        |
                                  5. SIP REFER
                                  (transferir a ext. agente)
                                        |
                                        v
                                  +----------+
                                  | Telefono |
                                  |  Agente  |
                                  | (ext 107)|
                                  +----------+
                                        |
                                  6. PBX puentea
                                  Visitante <-> Agente
                                        |
                                  7. Server BYE
                                  (desconecta su leg)
```

8.3.2 Flujo Detallado

1. El visitante proporciona su telefono.
2. El servidor valida y genera un `callId`.
3. Se origina un SIP INVITE al telefono del visitante desde la extension del chatbot.
4. Se gestiona autenticacion Digest (401/407) con Vozelia.
5. Al recibir `180 Ringing` o `183 Session Progress`, el estado pasa a `ringing`.
6. Al recibir `200 OK` (visitante responde), se envia ACK.
7. Se ejecuta REFER hacia las extensiones del ring group (secuencialmente: 107, 158, 105).
8. La PBX transfiere la llamada al primer agente que responde.
9. El leg del servidor se desconecta con BYE (la PBX mantiene la llamada visitante-agente).
10. CallerID en el telefono del agente: `"Lead Web" <telefono_visitante>`.

8.3.3 Configuracion del Ring Group

```
CLICK2CALL_EXTENSIONS=107,158,105
# 107 = Claudia
# 158 = David (oficina)
# 105 = David (casa)
```

Las extensiones se intentan secuencialmente. Si una falla (timeout de 15s en REFER), se prueba la siguiente.

8.3.4 Grabacion via PBX

Se solicita grabacion con el header SIP `Record: on` en el INVITE. La PBX (Vozelia) gestiona la grabacion en su lado.

---

9. Almacenamiento de Datos y Persistencia

9.1 Tablas PostgreSQL

Tabla	Registros Esperados	Proposito
`tenants`	~10	Clientes/organizaciones (multi-tenant)
`widget_themes`	~10	Temas visuales por tenant (JSONB completo)
`conversations`	Miles/mes	Sesiones de chat con FSM de estado
`messages`	Decenas miles/mes	Mensajes con tipos ricos y adjuntos
`leads`	Cientos/mes	Leads capturados con scoring
`agents`	~20	Usuarios agentes con roles y extensiones SIP
`calls`	Cientos/mes	Registros de llamadas (Click2Call + WebRTC)

`call_recordings`	Cientos/mes	Grabaciones con transcripciones (30 dias retencion)
`knowledge_entries`	~500	Base de conocimiento con embeddings vectoriales
`canned_responses`	~100	Respuestas predefinidas por shortcut
`webhooks`	~20	Endpoints de webhook configurados
`webhook_deliveries`	Miles/mes	Log de entregas de webhook
`analytics_events`	Decenas miles/mes	Eventos para graficas y metricas
`response_feedback`	Miles/mes	Feedback de calidad de respuestas del bot
`learned_responses`	Cientos	Respuestas aprendidas con embeddings
`callbacks`	Cientos/mes	Callbacks programados (Calendly-style)
`agent_notes`	Miles/mes	Notas internas por conversacion
`conversation_transfers`	Decenas/mes	Log de transferencias entre agentes
`scheduled_messages`	Decenas/mes	Mensajes programados para envio futuro
`file_uploads`	Cientos/mes	Archivos subidos (imagenes, PDFs)
`config`	~10	Configuracion key-value (JSONB)
`kb_scrape_log`	Decenas/mes	Historial de scraping automatico

9.2 Redis

Clave / Canal	Tipo	Proposito	TTL
`session:{visitorId}`	Hash	Sesion del visitante (idioma, BU, contexto)	24h
`ai:provider`	String	Proveedor de IA activo	Persistente
`crm:sf:token`	String	Token OAuth de Salesforce	3500s
`crm:zoho:token`	String	Token OAuth de Zoho	3500s

`proactive:{visitorId}:{trigger}`	String	Cooldown de trigger proactivo	Variable
`queue:{name}`	Sorted Set	Cola de llamadas por BU (score=timestamp)	24h
`queue:{name}:meta:{callerId}`	Hash	Metadatos del llamante en cola	24h
`queue:{name}:durations`	List	Ultimas 20 duraciones de llamadas	24h
`queue:agents:{bl}`	Set	Agentes online por BU	24h
`queue:agents:all`	Set	Todos los agentes online	24h

Canales Pub/Sub:

Canal	Publicador	Suscriptor	Proposito
`agent:message`	agent-handler	chat-handler	Mensaje agente -> visitante
`agent:typing`	agent-handler	chat-handler	Indicador de escritura agente
`visitor:message`	chat-handler	agent-handler	Mensaje visitante -> agente
`visitor:typing`	chat-handler	agent-handler	Indicador de escritura visitante
`message:read`	agent-handler	chat-handler	Confirmacion de lectura
`queue:new`	chat-handler	agent-handler	Nueva conversacion en cola
`call:incoming`	chat-handler	agent-handler	Llamada entrante
`call:accepted`	agent-handler	sip-signaling	Agente acepta llamada
`call:rejected`	agent-handler	sip-signaling	Agente rechaza llamada

9.3 Almacenamiento de Archivos

Directorio	Contenido	Retencion

`server/recordings/`	Grabaciones de llamadas (WAV/MJR)	30 dias (cron cada 6h)
`server/uploads/`	Archivos subidos por visitantes/agentes	Sin limite
`janus/recordings/`	Grabaciones Janus raw (MJR)	Importadas al server

9.4 Politica de Retencion de 30 Dias

El servicio `call-recording.js` ejecuta un cron automatico:

1. **Cada 6 horas**: Busca grabaciones con `started_at < 30 dias`.
2. Elimina archivos fisicos del disco.
3. Limpia campos `recording_url`, `recording_path`, `file_size_bytes`, `transcript`.
4. Anade nota: `"Cleaned after 30 days"`.
5. **Cada 90 dias**: Purga registros de metadatos completos.

---

10. Inteligencia Artificial y Aprendizaje

10.1 Arquitectura Multi-Proveedor

```
+-----------------------------------------------------+
|                    ai.js - Router                   |
+-----------------------------------------------------+
| getProvider()                                       |
|   -> Redis 'ai:provider' || env AI_PROVIDER        |
|                                                     |
| aiComplete(system, user, options)                   |
|   -> Intenta: primary                               |
|   -> Fallback: gemini -> anthropic -> openai        |
|   -> Si todos fallan: throw Error                   |
+-----------------------------------------------------+
|                                                     |
| anthropicComplete()   geminiComplete()  openai..() |
| - claude-sonnet-4     - gemini-2.5-flash  - gpt-4o-mini |
| - max_tokens: 2048    - max_tokens: 2048  - max_tokens: 2048 |
```

```
| - temperature: 0.4    - temperature: 0.4  - temperature: 0.4 |
+-----------------------------------------------------+
```

El proveedor activo se puede cambiar en caliente via Redis (`ai:provider`) sin reiniciar el servidor.

10.2 Knowledge Base y RAG

La base de conocimiento tiene tres fuentes:

1. **Archivos YAML** (`server/config/knowledge/*.yaml`): Cargados al inicio, sembrados en PostgreSQL.
2. **Auto-scraping** (`kb-scraper.js`): Extrae contenido de 11 paginas de redegal.com cada 24 horas.
3. **Entradas manuales**: Anadidas via API o dashboard.

Busqueda en 3 niveles:

```
Consulta del visitante
        |
        +---> Full-text search (PostgreSQL tsvector)
        |     Busqueda por palabras clave
        |
        +---> Vector search (pgvector, cosine similarity)
        |     Busqueda semantica con embeddings 768-dim
        |     Filtro: similarity > 0.5
        |
        +---> Learned responses search
              Busqueda en respuestas aprendidas
              Filtro: similarity > 0.6
        |
        v
   Contexto combinado -> System Prompt -> AI Provider
```

10.3 Auto-Scraping de redegal.com

Paginas monitorizadas:

URL	Categoria	BU
`/es/`	empresa	general
`/es/quienes-somos/`	empresa	general

`/es/servicios/`	servicios	general
`/es/servicios/boostic/`	producto	boostic
`/es/servicios/binnacle-data/`	producto	binnacle
`/es/servicios/marketing-digital/`	servicios	marketing
`/es/servicios/tech/`	servicios	tech
`/es/casos-de-exito/`	casos	general
`/es/blog/`	blog	general
`/es/contacto/`	contacto	general
`/es/accionistas-e-inversores/`	inversores	general

Proceso:

1. Fetch de la pagina con timeout de 15 segundos.
2. Extraccion de texto (elimina scripts, styles, nav, footer, HTML tags).
3. Calculo de hash SHA-256 del contenido.
4. Comparacion con el ultimo hash conocido.
5. Si cambio: actualizar o insertar entrada + generar embedding.
6. Rate limit: 2 segundos entre requests.

10.4 Aprendizaje desde CSAT

```
                CSAT >= 4
                    |
                    v
        +------------------------+
        | processCSATForLearning |
        +------------------------+
                    |
        Busca respuestas del bot
        no revisadas de esa
        conversacion
                    |
                    v
        +------------------------+
        |    learnResponse()     |
        |------------------------|
        | - Genera embedding     |
        | - Calcula confianza:   |
```

```
        |   CSAT 5 -> 0.9       |
        |   CSAT 4 -> 0.7       |
        | - Inserta en           |
        |   learned_responses    |
        +------------------------+
```

10.5 Ciclo de Feedback del Agente

1. Cada respuesta del bot se registra en `response_feedback`.
2. Los agentes pueden marcar respuestas como `good` o `bad`.
3. Si el agente corrige una respuesta, se aprende con confianza 0.95.
4. Si marca como `good`, se aprende con confianza 0.85.
5. Las respuestas aprendidas se incorporan al contexto RAG de futuras respuestas.

10.6 Lead Scoring Deterministico

Factor	Puntos	Maximo
Email proporcionado	20	20
Empresa proporcionada	15	15
Telefono proporcionado	10	10
Linea de negocio detectada	10	10
Mensajes intercambiados	2/msg	20
Escalacion a agente	15	15
Llamada VoIP solicitada	10	10
Total maximo		**100**

---

11. Concurrencia y Multi-Usuario

11.1 Gestion de Conexiones WebSocket

El servidor mantiene tres pools de WebSocket independientes:

Pool	Path	maxPayload	Proposito

`wssChat`	`/ws/chat`	16 KB	Visitantes del widget
`wssAgent`	`/ws/agent`	64 KB	Agentes del dashboard
`wssSip`	`/ws/sip`	16 KB	Senalizacion SIP/WebRTC

Cada pool mantiene un `Map` de conexiones activas:

- `visitors`: `visitorId` -> `WebSocket`
- `agents`: `agentId` -> `WebSocket`
- `participants`: `WebSocket` -> `{ role, callId, id }`

11.2 Rate Limiting WebSocket

```
const WS_RATE = { maxMessages: 20, windowMs: 10000 };
// 20 mensajes por cada 10 segundos por visitante
```

- Se limpian buckets expirados cada 60 segundos.
- Si se excede el limite, se envia error `Too many messages`.

11.3 Redis Pub/Sub para Notificaciones en Tiempo Real

Redis actua como broker de mensajes entre los tres pools de WebSocket:

```
Visitante A                          Agente 1
(wssChat)                            (wssAgent)
    |                                    |
    |  mensaje                           |
    +-------> chat-handler               |
                  |                      |
                  | Redis pub            |
                  | visitor:message      |
                  +---------> agent-handler
                                  |
                                  +---> Agente 1 WS
```

11.4 Gestion de Cola de Agentes

- Los agentes tienen un `max_concurrent` configurable (default: 3 conversaciones simultaneas).
- El claim de conversacion es atomico: `UPDATE ... WHERE agent_id IS NULL RETURNING *`.
- Cuando un agente cierra una conversacion, se verifica si tiene otras activas antes de cambiar su estado a `online`.

11.5 Cola de Llamadas con Posicion

El sistema `call-queue.js` proporciona:

- **Colas por BU**: Redis sorted sets (`queue-boostic`, `queue-binnacle`, etc.).
- **Posicion en cola**: O(log N) via `ZRANK`.
- **Tiempo estimado**: `(position * avgCallDuration) / onlineAgents`.
- **Agentes online por BU**: Redis sets para conteo rapido.
- **Historial de duraciones**: Ultimas 20 duraciones para calculo de media.
- **TTL de 24h**: Limpieza automatica de datos obsoletos.

---

12. Gestion de Leads y Notificaciones

12.1 Fuentes de Captura de Leads

Fuente	Campos Capturados	Trigger
Formulario de lead (widget)	nombre*, email, telefono, empresa	`lead_submit`
Formulario offline	nombre*, email*, telefono, mensaje	`offline_form` (fuera de horario)
Conversacion	datos mencionados en el chat	Extraccion automatica
Llamada	telefono del visitante	`request_call`

(*) = obligatorio

12.2 Ciclo de Vida del Lead

```
new -> contacted -> qualified -> converted
                                    |
                                    v
                                  lost
```

12.3 Despacho a CRM

CRM	Operaciones Soportadas	Autenticacion
Salesforce	createContact (Lead), createDeal (Opportunity), test	OAuth2 Client Credentials
HubSpot	createContact, createDeal, logActivity (Notes), test	Bearer Token (API Key)
Zoho	createContact (Lead), test	OAuth2 Refresh Token
Pipedrive	createContact (Person), createDeal, test	API Token

Todos los adaptadores mapean campos del lead a los campos nativos del CRM (nombre, email, telefono, empresa, fuente="Chatbot Widget", linea de negocio, idioma, quality score).

12.4 Sistema de Webhooks

11 eventos soportados:

Evento	Descripcion	Datos incluidos
`conversation.started`	Nueva conversacion	conversationId, visitorId, language, businessLine
`conversation.closed`	Conversacion cerrada	conversationId, agentId
`message.received`	Mensaje del visitante	conversationId, visitorId, content
`message.sent`	Mensaje del bot/agente	conversationId, sender, content
`lead.created`	Nuevo lead capturado	lead (completo), visitorId, businessLine

`lead.updated`	Lead actualizado	lead (completo)
`agent.assigned`	Agente asignado	conversationId, agentId, agentName
`agent.transferred`	Conversacion transferida	conversationId, fromAgentId, toAgentId, reason
`call.started`	Llamada iniciada	callId, conversationId, visitorId, phone, mode
`call.ended`	Llamada finalizada	callId, duration, reason
`csat.submitted`	Valoracion enviada	conversationId, visitorId, rating, comment

Formato del payload:

```
{
  "event": "lead.created",
  "data": { ... },
  "timestamp": "2026-02-21T10:30:00.000Z"
}
```

Header de firma HMAC:

```
X-Webhook-Signature: sha256=abc123def456...
```

Calculado como: `HMAC-SHA256(secret, JSON.stringify(payload))`

Reintentos con backoff exponencial:

Intento	Delay
1	Inmediato
2	5 segundos
3	10 segundos
4	20 segundos
5	40 segundos
6	80 segundos

Despues de 6 intentos fallidos, se incrementa `failure_count`. Los webhooks con `failure_count >= 50` se desactivan automaticamente.

12.5 Notificaciones por Email

El sistema envia tres tipos de email:

Tipo	Asunto	Contenido
Escalacion	`[Chatbot] Lead Web quiere hablar -- {BU}`	Alerta con ultimos 5 mensajes de contexto
Solicitud de llamada	`[Chatbot] Lead Web pide llamada -- {telefono} ({BU})`	Numero clickeable para devolver llamada
Resumen de conversacion	`[Chatbot] Resumen -- {BU} ({fecha})`	Transcripcion completa + datos del lead + metricas

Todos los emails usan plantillas HTML responsive con branding Redegal.

---

13. Funcionalidades del Dashboard

13.1 Inbox de Agente

- Lista de conversaciones pendientes (cola) con prioridad visual.
- Conversaciones activas asignadas al agente.
- Vista de mensajes con diferenciacion por tipo de sender (visitante, bot, agente, sistema, nota).
- Indicador de escritura del visitante en tiempo real.
- Confirmacion de lectura bidireccional.
- Contexto de pagina del visitante (URL actual, referrer).

13.2 Historial de Llamadas

- Lista paginada con filtros por BU y estado.
- Detalle de llamada: duracion, extension, estado, grabacion.
- Reproductor de grabaciones inline.
- Boton de transcripcion bajo demanda.
- Monitoring: supervisores pueden escuchar llamadas en curso.

13.3 Analytics

- Conversaciones por dia/semana/mes.
- Leads por linea de negocio.
- Tasa de conversion (leads/conversaciones).
- CSAT promedio por periodo y por agente.
- Tiempos de respuesta.
- Volumen por hora del dia.

13.4 Gestion de Leads

- Tabla de leads con filtros por estado, BU, score.
- Detalle del lead con historial de conversacion asociado.
- Cambio de estado: new -> contacted -> qualified -> converted/lost.
- Exportacion de datos.

13.5 Gestion de Equipo

- Lista de agentes con estado actual (online, busy, away, offline).
- Creacion de nuevos agentes con roles.
- Asignacion de idiomas y lineas de negocio.
- Configuracion de extension SIP.
- Limite de conversaciones simultaneas.

13.6 Settings

Seccion	Descripcion
Canned Responses	CRUD de respuestas predefinidas por shortcut, idioma, BU y categoria
Webhooks	Configuracion de endpoints, eventos suscritos, secreto HMAC, log de entregas
Theme	Personalizacion completa del widget (colores, tipografia, layout, features, sonidos)
System	Configuracion general (horario, IA, SIP, SMTP, CRM)

13.7 Wallboard (Tiempo Real)

Dashboard de supervision con actualizacion cada 5 segundos via WebSocket:

- **Global**: Llamadas activas, chats en cola, agentes online, SLA %.

- **Por BU**: Cola por linea de negocio, agentes asignados, tiempo de espera.
- **Agentes**: Estado de cada agente, llamadas y chats del dia, CSAT individual.
- **Today Stats**: Llamadas totales/respondidas/perdidas, chats resueltos, leads capturados, tasa de conversion.

---

14. Integraciones

14.1 Conectores CRM

Ver seccion 12.3 para detalles de cada adaptador.

Configuracion en la tabla `config` con clave `crm_integrations`:

```
[
  {
    "type": "hubspot",
    "enabled": true,
    "createDeal": true,
    "logActivities": true,
    "config": {
      "apiKey": "pat-xxxxx"
    }
  },
  {
    "type": "salesforce",
    "enabled": false,
    "config": {
      "instanceUrl": "https://mycompany.salesforce.com",
      "clientId": "...",
      "clientSecret": "..."
    }
  }
]
```

14.2 Sistema de Webhooks

Ver seccion 12.4 para detalles completos.

14.3 Plugin WordPress

Archivo: `plugins/wordpress/redegal-chatbot.php`

Caracteristicas:

- Panel de configuracion completo en Settings > Redegal Chatbot.
- Opciones: Server URL, API Key, idioma, color primario, posicion, logo, nombre empresa.
- Restriccion por paginas (patrones con fnmatch).
- Toggle on/off global.
- Inyeccion automatica del loader.js en el footer.

Instalacion:

1. Copiar `redegal-chatbot.php` a `wp-content/plugins/redegal-chatbot/`.
2. Activar en Plugins.
3. Configurar en Settings > Redegal Chatbot.

14.4 Snippet Shopify

Archivo: `plugins/shopify/assets/redegal-chatbot.liquid`

Caracteristicas:

- Snippet Liquid renderizable con `{% render 'redegal-chatbot' %}`.
- Configuracion via Theme Settings (settings_schema.json incluido).
- Usa `{{ shop.name }}` como nombre de empresa automaticamente.
- Toggle habilitado/deshabilitado desde Theme Editor.

Instalacion:

1. Subir `redegal-chatbot.liquid` a la carpeta `assets/` del tema.
2. Importar `settings_schema.json` en la configuracion del tema.
3. Anadir `{% render 'redegal-chatbot' %}` antes de `</body>` en `theme.liquid`.
4. Configurar desde Shopify Admin > Theme > Customize.

14.5 Plantilla Magento 2

Archivo:
`plugins/magento/view/frontend/templates/chatbot.phtml`

Caracteristicas:

- Template PHTML para Magento 2.
- Configuracion via Stores > Configuration > Redegal > Chatbot.
- Soporte multi-store.
- Inyeccion automatica en `before.body.end`.

Instalacion:

1. Crear modulo en `app/code/Redegal/Chatbot/`.
2. Configurar layout XML para inyectar el template.
3. Configurar desde Stores > Configuration > Redegal > Chatbot.

14.6 API de Embedding Manual del Widget

Para integracion directa sin plugin:

```
<script>
  window.RedegalChatbot = {
    baseUrl: 'https://chatbot.redegal.com/widget',
    apiUrl: 'wss://chatbot.redegal.com/ws/chat',
    configUrl:
'https://chatbot.redegal.com/api/config/widget',
    language: 'auto',
    primaryColor: '#007fff',
    theme: {
      branding: {
        companyName: 'Mi Empresa',
        logoUrl: 'https://example.com/logo.png'
      },
      layout: {
        position: 'bottom-right'
      }
    }
  };
</script>
```

```
<script src="https://chatbot.redegal.com/widget/loader.js"
async></script>
```

Opciones configurables en `window.RedegalChatbot`:

Opcion	Tipo	Default	Descripcion
`baseUrl`	string	-	URL base del widget
`apiUrl`	string	-	URL WebSocket del chat
`configUrl`	string	-	URL de configuracion del widget
`language`	string	`"auto"`	Idioma: `auto`, `es`, `en`, `pt`, `gl`
`primaryColor`	string	`"#007fff"`	Color primario
`theme.branding.companyName`	string	`"Redegal"`	Nombre de la empresa
`theme.branding.logoUrl`	string	`""`	URL del logo
`theme.layout.position`	string	`"bottom-right"`	Posicion: `bottom-right`, `bottom-left`

---

15. Seguridad

15.1 Autenticacion JWT

- **Algoritmo**: HS256
- **Expiracion**: 12 horas
- **Payload**: `{ id, username, displayName, role }`
- **En produccion**: `JWT_SECRET` es obligatorio (falla al iniciar sin el).
- **En desarrollo**: Se genera automaticamente un secreto temporal.

Roles soportados:

Rol	Permisos
`agent`	Atender conversaciones, cerrar chats, usar canned responses
`supervisor`	Todo de agent + monitoring de llamadas + wallboard

`admin`	Todo de supervisor + gestion de equipo + configuracion
`architect`	Acceso completo incluyendo configuracion tecnica
`developer`	Acceso a APIs de integracion
`qa`	Revision de respuestas y feedback

15.2 Validacion de API Key

Los endpoints del widget requieren `X-API-Key` header o parametro `apiKey`:

```
function requireApiKey(req, res, next) {
  const key = req.headers['x-api-key'] || req.query.apiKey;
  if (!key || key !== process.env.WIDGET_API_KEY) {
    return res.status(403).json({ error: 'Invalid API key' });
  }
  next();
}
```

En desarrollo con `DEV_SKIP_AUTH=true`, se permite acceso sin API key.

15.3 Politica CORS

```
ALLOWED_ORIGINS=https://redegal.com,https://www.redegal.com,http
```

- Se valida el header `Origin` tanto en HTTP como en WebSocket upgrades.
- En produccion, las conexiones WebSocket desde origenes no permitidos se rechazan con `403 Forbidden`.

15.4 Headers de Seguridad (CSP y Mas)

Header	Valor
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`SAMEORIGIN`

`Referrer-Policy`	`strict-origin-when-cross-origin`
`X-XSS-Protection`	`0` (se usa CSP en su lugar)
`Permissions-Policy`	`camera=(), microphone=(self), geolocation=()`
`X-Permitted-Cross-Domain-Policies`	`none`
`Cross-Origin-Opener-Policy`	`same-origin`
`Cross-Origin-Resource-Policy`	`same-origin`
`X-DNS-Prefetch-Control`	`off`
`Origin-Agent-Cluster`	`?1`
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload` (solo produccion)

Content Security Policy (CSP):

```
default-src 'self';
script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
font-src 'self' https://fonts.gstatic.com;
connect-src 'self' wss: ws: [allowed-origins];
img-src 'self' data: https:;
media-src 'self' blob:;
frame-ancestors 'self' [allowed-origins]
```

15.5 Rate Limiting

Contexto	Limite	Ventana
WebSocket (visitante)	20 mensajes	10 segundos
Express JSON body	100 KB max	Por request
Webhooks failure	50 fallos	Desactivacion automatica

15.6 Sanitizacion de Input (Prevencion XSS)

```
// En chat-handler.js - todo mensaje de visitante:
msg.content = msg.content
  .slice(0, 2000)                              // Max 2000
caracteres
  .replace(/[\x00-\x08\x0B\x0C\x0E-\x1F]/g, '') // Strip
caracteres de control
  .replace(/<[^>]*>/g, '');                     // Strip tags
HTML (previene stored XSS)

// En email.js - todo contenido en plantillas HTML:
function escapeHtml(str) {
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;');
}
```

15.7 Firmas HMAC de Webhooks

```
const signature = crypto
  .createHmac('sha256', webhook.secret)
  .update(payload)
  .digest('hex');
headers['X-Webhook-Signature'] = `sha256=${signature}`;
```

El receptor debe verificar la firma comparando:

```
HMAC-SHA256(secret, request_body) === valor_del_header
```

15.8 Proteccion CSRF

Implementacion basada en validacion de Origin header (sin tokens):

1. Metodos seguros (`GET` , `HEAD` , `OPTIONS`): siempre permitidos.
2. Requests con `X-API-Key` : exentos (machine-to-machine).

3. Requests con `X-Requested-With` : exentos (AJAX con custom header).
4. Sin `Origin` / `Referer` pero con `Content-Type: application/json` : permitidos (browsers envian `form-urlencoded` para CSRF).
5. En cualquier otro caso: se valida `Origin` contra `ALLOWED_ORIGINS` .

15.9 Prevencion SSRF en Webhooks

```
function isAllowedUrl(urlStr) {
  // Bloquea: localhost, 127.0.0.1, ::1, 0.0.0.0
  // Bloquea: redes privadas (10.x, 172.16-31.x, 192.168.x)
  // Bloquea: metadata AWS (169.254.169.254)
  // Bloquea: .local, .internal
  // Solo permite: http:, https:
}
```

---

16. Guia de Despliegue en Produccion

16.1 Configuracion Docker Compose

El archivo `docker-compose.yml` del proyecto define 4 servicios:

```
services:
  postgres:       # pgvector/pgvector:pg16 — Puerto 5432 (solo
localhost)
  redis:          # redis:7-alpine — Puerto 6379 (solo
localhost)
  server:         # Build desde ./server — Puerto 9456 (solo
localhost)
  janus:          # canyan/janus-gateway — Puertos 8088, 8188,
7088, 10000-10200/udp
```

Importantes:

- Todos los puertos estan vinculados a `127.0.0.1` excepto Janus (necesita acceso desde el navegador del visitante).
- PostgreSQL inicializa automaticamente con `init.sql`.
- El server depende de que postgres y redis esten healthy.
- Los builds del widget y dashboard se montan como volumenes read-only.

16.2 Configuracion Nginx Reverse Proxy

```
upstream chatbot_backend {
    server 127.0.0.1:9456;
}

server {
    listen 443 ssl http2;
    server_name chatbot.redegal.com;

    ssl_certificate
/etc/letsencrypt/live/chatbot.redegal.com/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/chatbot.redegal.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    # HTTP -> HTTPS redirect
    add_header Strict-Transport-Security "max-age=63072000;
includeSubDomains; preload" always;

    # Proxy principal
    location / {
        proxy_pass http://chatbot_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # WebSocket endpoints
    location ~ ^/ws/(chat|agent|sip)$ {
        proxy_pass http://chatbot_backend;
```

```
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 86400;
    }

    # Janus WebSocket (acceso publico para widget WebRTC)
    location /janus-ws {
        proxy_pass http://127.0.0.1:8188;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 86400;
    }

    # Limitar tamano de uploads
    client_max_body_size 10M;
}

server {
    listen 80;
    server_name chatbot.redegal.com;
    return 301 https://$server_name$request_uri;
}
```

16.3 SSL/TLS

Recomendado: Let's Encrypt con certbot:

```
sudo certbot --nginx -d chatbot.redegal.com
```

16.4 Variables de Entorno para Produccion

```
# OBLIGATORIAS en produccion
NODE_ENV=production
JWT_SECRET=cadena_aleatoria_segura_de_64_caracteres
POSTGRES_PASSWORD=password_seguro_generado
```

```
REDIS_PASSWORD=otro_password_seguro
WIDGET_API_KEY=api_key_generada

# CORS — dominio real
ALLOWED_ORIGINS=https://redegal.com,https://www.redegal.com,http

# IP publica del servidor (para NAT traversal de Janus)
SERVER_PUBLIC_IP=37.27.92.122

# Janus WebSocket publico (a traves de Nginx)
JANUS_PUBLIC_WS=wss://chatbot.redegal.com/janus-ws

# IA
AI_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=AIza...

# SIP
SIP_DOMAIN=cloudpbx1584.vozelia.com
SIP_EXTENSION=108
SIP_PASSWORD=...
CLICK2CALL_EXTENSIONS=107,158,105

# SMTP
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=chatbot@redegal.com
SMTP_PASSWORD=app_password
NOTIFICATION_EMAIL=equipo@redegal.com
```

16.5 Requisitos del Servidor

Recurso	Minimo	Recomendado
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Disco	20 GB SSD	40 GB SSD
SO	Ubuntu 22.04+	Ubuntu 24.04 LTS
Docker	24.x	Ultima version estable
Red	100 Mbps	1 Gbps

16.6 Monitoring y Health Checks

Endpoint de Health:

```
curl https://chatbot.redegal.com/health
# Respuesta: { "status": "ok", "uptime": 123456 }
```

Docker health checks (configurados en docker-compose.yml):

Servicio	Check	Intervalo
PostgreSQL	`pg_isready`	5s
Redis	`redis-cli ping`	5s

Procesos internos de mantenimiento:

Proceso	Frecuencia	Descripcion
Limpieza de grabaciones	Cada 6 horas	Elimina archivos > 30 dias
Scraping de knowledge base	Cada 24 horas	Actualiza KB desde redegal.com
Embedding de nuevas entradas	Al iniciar	Genera embeddings para entradas sin vector
Limpieza de sesiones stale en Janus	Cada 5 minutos	Elimina sesiones > 1 hora
Limpieza de rate limit buckets	Cada 60 segundos	Elimina buckets expirados

16.7 Comandos de Despliegue

```
# Despliegue completo desde cero
docker compose up --build -d

# Actualizar solo el servidor (tras git pull)
docker compose build server && docker compose up -d server

# Ver logs
docker compose logs -f server
docker compose logs -f postgres
docker compose logs -f janus
```

```
# Reiniciar un servicio
docker compose restart server

# Backup de base de datos
docker compose exec postgres pg_dump -U redegal
redegal_chatbot > backup.sql

# Restaurar base de datos
docker compose exec -T postgres psql -U redegal
redegal_chatbot < backup.sql
```

Apendice A: Endpoints de la API REST

Metodo	Ruta	Auth	Descripcion
GET	`/health`	-	Estado del servidor
GET	`/api/config/widget`	API Key	Configuracion del widget
POST	`/api/chat`	API Key	Enviar mensaje (HTTP alternativo a WS)
GET	`/api/leads`	JWT	Listar leads
POST	`/api/leads`	JWT	Crear lead manualmente
GET	`/api/agents`	JWT	Listar agentes
POST	`/api/agents/login`	-	Login de agente (retorna JWT)
GET	`/api/calls`	JWT	Historial de llamadas
GET	`/api/callbacks`	JWT	Listar callbacks programados
GET	`/api/analytics`	JWT	Datos de analytics
POST	`/api/upload`	JWT	Subir archivo

GET	`/api/settings`	JWT (admin)	Obtener configuracion
PUT	`/api/settings`	JWT (admin)	Actualizar configuracion
GET	`/api/training`	JWT	Listar respuestas para revision
POST	`/api/training/:id/feedback`	JWT	Enviar feedback de respuesta
GET	`/api/wallboard`	JWT	Datos del wallboard
GET	`/api/conversations/:id/transcript`	JWT	Exportar transcripcion (JSON/HTML/TXT)

Apendice B: Mensajes WebSocket

Canal `/ws/chat` (Visitante)

Mensajes del cliente:

type	Datos	Descripcion
`chat`	`{ content }`	Enviar mensaje de chat
`set_language`	`{ language }`	Cambiar idioma
`set_business_line`	`{ businessLine }`	Cambiar linea de negocio
`lead_submit`	`{ name, email, phone, company }`	Enviar formulario de lead
`offline_form`	`{ name, email, phone, message }`	Formulario fuera de horario
`escalate`	`{}`	Solicitar agente humano
`request_call`	`{ phone }`	Solicitar callback
`request_webrtc_call`	`{}`	Solicitar llamada WebRTC

`webrtc_hangup`	`{ callId, duration }`	Colgar llamada WebRTC
`csat`	`{ rating, comment }`	Enviar valoracion
`page_context`	`{ pageUrl, pageTitle, ... }`	Enviar contexto de pagina
`search_kb`	`{ query, businessLine }`	Buscar en knowledge base
`quick_reply`	`{ value }`	Seleccionar quick reply
`visitor_typing`	`{ isTyping }`	Indicador de escritura
`request_transcript`	`{}`	Solicitar transcripcion

Mensajes del servidor:

type	Datos	Descripcion
`connected`	`{ visitorId, isBusinessHours }`	Conexion establecida
`chat_history`	`{ messages[], language, businessLine, state }`	Historial previo
`message`	`{ sender, content, richContent, timestamp }`	Mensaje nuevo
`typing`	`{ isTyping }`	Bot esta escribiendo
`language_detected`	`{ language }`	Idioma auto-detectado
`business_line_detected`	`{ businessLine }`	BU detectada por URL
`escalating`	`{ message }`	Buscando agente
`call_initiated`	`{ callId, message }`	Llamada en curso
`webrtc_ready`	`{ callId, janusWsUrl, sipProxy, ... }`	Config para WebRTC
`proactive_message`	`{ trigger, content }`	Mensaje proactivo
`kb_results`	`{ query, results[] }`	Resultados de busqueda KB
`queue_update`	`{ position, estimatedWait }`	Posicion en cola

Canal `/ws/agent` (Agente)

Mensajes del cliente:

type	Datos	Descripcion
`accept_conversation`	`{ conversationId }`	Aceptar conversacion
`send_message`	`{ conversationId, content }`	Enviar mensaje (soporta /shortcuts)
`close_conversation`	`{ conversationId }`	Cerrar conversacion
`transfer_conversation`	`{ conversationId, targetAgentId, reason }`	Transferir
`internal_note`	`{ conversationId, content }`	Nota interna
`set_status`	`{ status }`	Cambiar estado del agente
`mark_read`	`{ conversationId }`	Marcar como leido
`typing`	`{ conversationId, isTyping }`	Indicador de escritura
`add_tags`	`{ conversationId, tags[] }`	Anadir etiquetas
`set_priority`	`{ conversationId, priority }`	Establecer prioridad
`get_suggestions`	`{ conversationId }`	Obtener sugerencias IA
`accept_call`	`{ callId }`	Aceptar llamada
`reject_call`	`{ callId }`	Rechazar llamada
`hangup_call`	`{ callId }`	Colgar llamada

Apendice C: Glosario

Termino	Definicion
BU	Business Unit — Linea de negocio (boostic, binnacle, marketing, tech)
CSAT	Customer Satisfaction Score — Valoracion del visitante (1-5 estrellas)

FSM	Finite State Machine — Maquina de estados de la conversacion
RAG	Retrieval-Augmented Generation — Enriquecimiento del prompt con datos de la KB
KB	Knowledge Base — Base de conocimiento del chatbot
Click2Call	Llamada de devolucion: el servidor llama al visitante y lo conecta con un agente
WebRTC	Web Real-Time Communication — Llamadas de audio directamente desde el navegador
REFER	Metodo SIP para transferir una llamada a otra extension
PBX	Private Branch Exchange — Centralita telefonica (Vozelia Cloud)
pgvector	Extension de PostgreSQL para busqueda de similitud con vectores
Shadow DOM	Tecnologia web para encapsular CSS del widget sin conflictos con la pagina host
IIFE	Immediately Invoked Function Expression — Formato del bundle del widget
MJR	Janus Media Recording — Formato propietario de grabacion de Janus

---

Documento generado para Redegal — A Smart Digital Company. Todos los derechos reservados. Febrero 2026.